Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 9

Watching System Calls with strace

Track A: The Real OS, from the Outside (Linux)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1. Introduction 3

2. Environment and Preparation 3

3. Part 1 - Watching the System Calls of ls 3

   3.1. Trace and Recorded Calls 3

   3.2. Call Counts and Observation 3

4. Part 2 - A C Program that Calls the Kernel 4

   4.1. Implementation of syscalls.c 4

   4.2. Compilation and Execution 4

   4.3. strace Analysis 5

   4.4. File Descriptors - Observation 5

5. Part 3 - Counting openat Calls 5

   5.1. Statistical Summary and Comparison 5

6. Verification and Conclusion 6

   6.1. Final File Verification 6

   6.2. Conclusion 6

1. Introduction

The purpose of this laboratory work was to observe how ordinary Linux programs cross the boundary between user space and the kernel through system calls. I used **strace** to inspect a standard **ls -l** command, implemented a small C program that performs file I/O with **open()**, **write()**, **read()**, and **close()**, and counted selected calls.

The experiments were performed in an Ubuntu virtual machine. All command outputs and numeric results in this report are taken from the recorded Lab 9 terminal session, rather than from sample values in the lab instructions.

2. Environment and Preparation

I created **~/os-lab9** and confirmed that the tracing utility was available. The installed version was **strace 6.19**. The C source was later compiled with GCC using the warning options required by the assignment.

3. Part 1 - Watching the System Calls of ls

3.1. Trace and Recorded Calls

I first ran the exact tracing command from the lab sheet. The **-f** option follows processes, while **-e trace=...** restricts the output to the four selected system-call types. The trace showed library loading, reads of Linux system files, opening the current directory, and writing the listing to standard output.

```
$ strace -f -e trace=openat,read,write,close ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(3) = 0
write(1, "total 0\n", 8) = 8
+++ exited with 0 +++
```

I then saved the trace to **part1-trace.txt** and the command output to **part1-output.txt**. When that second listing was recorded, the folder already contained the two newly created trace/output files; consequently, its listing began with **total 4** instead of the original **total 0**.

```
$ strace -f -e trace=openat,read,write,close \
    -o part1-trace.txt ls -l | tee part1-output.txt
$ grep 'write(1,' part1-trace.txt
6499  write(1, "total 4\n-rw-rw-r-- 1 alex_007m a"..., 143) = 143
```

3.2. Call Counts and Observation

The saved trace was counted with **grep -c** for each call type. It contained the following observed calls:

openat()	31
read()	20
write()	1
close()	34
Total of these four types	**86**

Observe: The **ls** command prints to the screen using **write()** with file descriptor **1**, the process standard output (**stdout**). In the saved run there was one **write(1,...)** for the listing.

4. Part 2 - A C Program that Calls the Kernel

4.1. Implementation of syscalls.c

I created **syscalls.c** following the provided code. It opens **note.txt** for writing, sends the message **"hello, kernel\n"**, closes the descriptor, reopens the file read-only, and reads the bytes back. The file itself is accessed with file-operation system calls rather than with **fprintf()** or **fscanf()**.

```
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    const char *msg = "hello, kernel\n";
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

The string includes a newline, so its 13 visible characters plus the newline occupy **14 bytes**. The file is created with permission mode **0644**, and the initial open uses **O_TRUNC** to replace any older contents.

4.2. Compilation and Execution

The compilation command produced the executable **syscalls** without reported compiler errors. Running the program returned descriptor 3 and confirmed that the full 14-byte message was both written and read.

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel

$ cat note.txt
hello, kernel
```

A file listing independently confirmed that **note.txt** existed and had a size of **14 bytes**. The program output was also saved to **part2-output.txt** for the report.

4.3. strace Analysis

Next, I traced **syscalls** with **strace -e trace=openat,write,read,close**. In this environment the C-library **open()** calls appeared as Linux **openat()** calls. After the initial loader operations, the following sequence was observed for **note.txt**:

```
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY) = 3
read(3, "hello, kernel\n", 63)        = 14
close(3)                                = 0
```

The trace also contained **write(1,...)** calls for the diagnostic messages printed by **printf()**. These are separate from **write(3,...)**, which writes the data to **note.txt**. The returns **= 14** confirm the transfer size, and **close(3) = 0** indicates successful descriptor closure.

4.4. File Descriptors - Observation

Observe: The first **openat()** returned descriptor **3** because **0** is normally standard input (**stdin**), **1** is standard output (**stdout**), and **2** is standard error (**stderr**). Those three descriptors are already open in this process, so the kernel assigns the lowest free descriptor, which is **3**. When it is closed, the subsequent file open can reuse that number.

5. Part 3 - Counting openat Calls

5.1. Statistical Summary and Comparison

The laboratory overview names a call-counting part. I used **strace -f -c -e trace=openat ls -l** to count this single type of system call. The **-c** option prints a statistical summary, while **-e trace=openat** keeps the tally focused on one syscall.

```
$ strace -f -c -e trace=openat ls -l
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- --------
100.00    0.001687          54        31         1 openat
------ ----------- ----------- --------- --------- --------
100.00    0.001687          54        31         1 total
```

A second run saved the statistics to **part3-count.txt**. That run also counted **31 openat() calls**, with **one failed call**. The recorded elapsed time and average microseconds per call changed (0.002084 s and 67 microseconds/call), but the call count stayed the same.

```
$ cat part3-count.txt
100.00    0.002084          67        31         1 openat
100.00    0.002084          67        31         1 total
$ grep -c 'openat(' part1-trace.txt
31
```

The count from **strace -c** therefore matched the independently counted **31 openat()** entries in **part1-trace.txt**. The one recorded error was an **ENOENT** for an optional locale file; **ls** nevertheless completed successfully with exit status **0**.

6. Verification and Conclusion

6.1. Final File Verification

At the end of the session I listed the working directory. The evidence shows that the source, compiled program, test file, trace files, saved statistical summary, and Markdown report were present.

```
$ cd ~/os-lab9 && ls -l
-rw-r--r--  ...    14 ... note.txt
-rw-rw-r--  ...   143 ... part1-output.txt
-rw-rw-r--  ...  5855 ... part1-trace.txt
-rw-rw-r--  ...    80 ... part2-output.txt
-rw-rw-r--  ...   779 ... part2-trace.txt
-rw-rw-r--  ...   310 ... part3-count.txt
-rw-rw-r--  ...   605 ... part3-output.txt
-rw-rw-r--  ...  2843 ... report.md
-rwxrwxr-x  ... 16264 ... syscalls
-rw-rw-r--  ...   976 ... syscalls.c
```

The most important evidence was the successful **syscalls** output, the **write(3,...)** and **read(3,...)** entries for **note.txt**, and the agreement of both **openat()** counts. The trace/output files were generated by shell redirection and **tee**, while **report.md** recorded the answers to the exercise questions.

6.2. Conclusion

In Laboratory Work No. 9, I used **strace** to inspect actual Linux system calls made by a familiar command and by a small C program. The trace made the roles of **openat()**, **read()**, **write()**, and **close()** directly visible, including the distinction between writing to a regular file and printing to the terminal.

The custom program successfully wrote and read a 14-byte message through descriptor 3. I also observed that one **ls -l** run generated 31 **openat()**, 20 **read()**, one **write()**, and 34 **close()** calls, and that a separate **strace -c** run confirmed the same 31 **openat()** calls. These experiments helped clarify how programs request file operations from the Linux kernel.